

ROD-256: A Cloud-Based Authentication System with a C++ Experimental Hash Algorithm

Rodina Amr Mukhtar

Bilgisayar Mühendisliği Bölümü, Kastamonu Üniversitesi

Kastamonu, Türkiye

 0009-0002-0696-0929

Abstract—This paper presents ROD-256, an original hash algorithm designed from scratch using object-oriented C++, and a full-stack cloud-based authentication system in which this algorithm serves as a core component. ROD-256 uses eight 32-bit internal state values, 64 mixing rounds, XOR operations, bit rotation (rotateLeft/rotateRight), and modular addition to produce a 256-bit digest. A five-stage mixing process is applied per input byte, and output sensitivity is targeted through multi-stage diffusion. The system incorporates RSA-signed JWT authentication, bcrypt password hashing, Supabase PostgreSQL cloud database integration, a hash-chained audit ledger, and a structural comparison component with SHA/RIPMD family algorithms. The front-end, developed with React and Vite, is deployed on Vercel; the back-end, built with Node.js and Express.js, is hosted on the Render cloud platform.

Keywords—hash algorithm; experimental authentication; C++ systems programming; RSA-JWT; hash-chained audit ledger; cloud security; ROD-256; bit rotation; XOR diffusion

I. INTRODUCTION

The rapid proliferation of digital systems and the pervasive integration of online services into daily life have made the security of user authentication systems critically important. Although password-based authentication remains the most common method, security vulnerabilities in the design of hash algorithms can provide fertile ground for data breaches [1]. In this context, there is a pressing need for an integrated approach that addresses both the cryptographic and engineering dimensions of authentication infrastructures.

This work was developed within a systems programming course at the Department of Computer Engineering, Kastamonu University, and produced an original hash algorithm named ROD-256. The algorithm was built from scratch using object-oriented C++ and integrated with a Node.js back-end via a compiled executable. The system encompasses a comprehensive cloud architecture spanning a React front-end hosted on Vercel, an Express.js back-end deployed on Render, and a Supabase PostgreSQL cloud database.

The primary contributions of this work are as follows:

- An original hash algorithm (ROD-256) designed in object-oriented C++ employing a five-stage mixing round is presented.
- A full-stack authentication system integrating systems programming with modern cloud architecture has been implemented.
- ROD-256 has been structurally evaluated against the SHA family and RIPEMD-160 algorithms.
- A hash-chained audit ledger supporting cryptographic integrity verification for every authentication event has been designed.
- An interactive, publicly accessible web application featuring a live animated visualization of ROD-256 has been deployed.

II. RELATED WORK

The cryptographic design of hash algorithms and the security mechanisms of authentication systems have been extensively studied in the literature. Early algorithms such as MD5 and SHA-1 have been demonstrated to be vulnerable to various attacks; SHA-2 and SHA-3 were subsequently standardized by NIST to address these weaknesses [2].

In the domain of password storage, key derivation functions such as bcrypt [3], scrypt [4], and Argon2 [5] provide comprehensive resistance to brute-force attacks through memory-intensive hashing structures. Bhatt and Patil [6] empirically demonstrated the contribution of multi-factor authentication layers to web application authentication security.

JWT-based authentication mechanisms are widely adopted for stateless session management [7]. The RS256 signing scheme, combined with RSA asymmetric cryptography, offers a substantially stronger security guarantee than HS256 [8]. The use of hash-chained audit ledgers for application log verification is attracting growing interest; by storing each entry's predecessor digest, retroactive tampering can be detected [9]. The ROD-256 system extends this tradition by realizing an experimental algorithm design on a full-stack platform [10].

III. SYSTEM ARCHITECTURE

The ROD-256 Experimental Hash Algorithm Authentication System consists of four principal layers that communicate with one another: the front-end client, the back-end API, the C++ hash engine, and the cloud data storage layer.

A. Cloud Architecture

The front-end is served as a static React application over the Vercel CDN infrastructure. Authentication requests are routed to the Node.js/Express.js back-end running on the Render platform. The back-end invokes the compiled C++ CLI engine for hashing operations and persists session data in Supabase PostgreSQL.

B. Authentication Flow

The system flow consists of seven steps:

1. The user accesses the React front-end.
2. A registration or login request is forwarded to the Render back-end.
3. Passwords are hashed with bcrypt before being stored in the database.
4. Upon successful login, a JWT signed with the RSA private key is generated.
5. The authentication event is hashed by ROD-256 and written to a hash-chained audit ledger block.
6. Data is persisted to Supabase PostgreSQL.
7. The user can verify the ledger and test hash algorithms.

IV. THE ROD-256 HASH ALGORITHM

A. General Design Philosophy

ROD-256 is an education-oriented algorithm that reinterprets the fundamental cryptographic principles of the SHA family—particularly the ARX (Add-Rotate-XOR) structure—within an original framework. In addition to the 256-bit output size shared with SHA-256 and the Nothing-up-my-sleeve (NUMS) constant-selection principle, ROD-256 adopts a serial processing model that applies independent multi-round mixing per character. This choice foregrounds implementation transparency and pedagogical traceability.

B. Data Structure and State Representation

The fundamental data structure of the algorithm is an eight-element `uint32_t` array defined within the `Rod256Hasher` class. Each element is 32 bits wide; the eight elements together constitute $8 \times 32 = 256$ bits—the digest size of the algorithm. The state array is initialized with NUMS constants derived from the hexadecimal expansion of π :

```
// rod256.cpp – resetState()
state = {
    0x243F6A88, 0x85A308D3,
    0x13198A2E, 0x03707344,
    0xA4093822, 0x299F31D0,
    0x082EFA98, 0xEC4E6C89
};
```

C. Round Constant Generation

A unique constant is generated for each round. This constant is computed by XORing the hexadecimal representation of the Golden Ratio, `0x9E3779B9`, with a round-index-dependent multiplier:

```
// rod256.cpp – roundConstant(int round)
return 0x9E3779B9 ^ (round * 0x7F4A7C15);
```

This approach aims to produce distinct constant values for each round and contribute to the differentiation of round outputs. The constant `0x9E3779B9` is a well-known cryptographic value also used in established block ciphers such as TEA and Blowfish, exploiting the unpredictable nature of irrational numbers.

D. Five-Stage Mixing Round — `mixCharacter()`

The `mixCharacter()` function, which forms the heart of ROD-256, executes 64 times per input byte, applying five distinct operations in each round. The full C++ implementation is given below:

```
// rod256.cpp – mixCharacter(uint32_t value)
for (int round = 0; round < ROUNDS; round++) {
    int index = round % STATE_SIZE;

    // Step 1: Initial XOR mixing
    uint32_t mix = state[index]
        ^ value
        ^ roundConstant(round);

    // Step 2: Left-rotation diffusion
    mix = rotateLeft(mix, 7);

    // Step 3: Neighboring state addition
    mix = mix + rotateRight(
        state[(index+1) % STATE_SIZE],
        11);
```

```
// Step 4: Distant state XOR diffusion
mix = mix ^ rotateLeft(
    state[(index+3) % STATE_SIZE],
    17);
```

```
// Step 5: Double-layer state update
state[index] = state[index] ^ mix;
state[index] = state[index]
    + rotateLeft(value + round,
        (round % 31) + 1);
}
```

The role of each step is explained below:

- Step 1 — Initial XOR Mixing: The current state value (`state[index]`), the input byte (`value`), and the round constant are combined via XOR. This triple XOR injects input information into the state.
- Step 2 — Left Rotation (`rotL 7`): The mix value is rotated 7 bits to the left. Bit rotation provides lossless bit-level positional shifting and supports non-linear diffusion.
- Step 3 — Neighboring State Addition: The next state cell (`index+1`) is right-rotated by 11 bits and added modularly. This creates a cross-mixing effect linking adjacent state cells.
- Step 4 — Distant State XOR Diffusion: The state cell three positions ahead (`index+3`) is left-rotated by 17 bits and XORed. This establishes long-range dependencies within a single round, contributing to the avalanche effect.
- Step 5 — Double-Layer Update: The state cell is first XORed with `mix`, then incremented by the input sum rotated by a round-dependent variable amount (`round % 31 + 1`). The variable rotation amount breaks the periodicity present in fixed-rotation designs.

E. Digest Generation — `hash()`

After all input bytes have been processed, the eight state values are converted to hexadecimal strings and concatenated. The process is as follows:

```
// rod256.cpp – hash()
std::string data = message + salt;
for (char ch : data) {
    uint32_t val =
        static_cast<uint32_t>(
            static_cast<unsigned char>(ch));
    mixCharacter(val);
}
// Digest assembly
for (uint32_t part : state) {
    result << hex << setw(8)
        << setfill('0') << part;
}
```

The message and salt are first concatenated to form a single data stream. Each character is converted to its byte value (0–255), and the use of unsigned char prevents sign-extension issues. Each of the eight state values maps to 8 hexadecimal characters (32 bits / 4 bits = 8), producing a total 64-character, 256-bit digest.

F. Salt Mechanism and Security Impact

ROD-256 mandates a salt parameter by design. Message and salt are concatenated and processed as a single data stream. This

approach has two important effects: (1) The same message produces different digests with different salts, neutralizing precomputed dictionary attacks (rainbow tables). (2) Selecting a unique salt per user ensures that different users sharing the same password have different digests in the database.

G. Example Output

For the following input pair, the ROD-256 digest is:

Message : password123

Salt : ABC987

ROD-256:

fbd11d248c0f19adf7efab23ed0b92b8

db8bff25c00b34afbfc1cf3843a8ea9d

64 hexadecimal characters $\times$ 4 bits = 256 bits. The eight 32-bit state values correspond respectively to the following hex blocks: fbd11d24, 8c0f19ad, f7efab23, ed0b92b8, db8bff25, c00b34af, bfc1cf38, 43a8ea9d.

V. SECURITY COMPONENTS

A. RSA-Signed JWT Authentication

The system performs token signing using the RS256 (RSA + SHA-256) algorithm. An RSA key pair is generated via the scripts/generateKeys.js script; the private key signs the token and the public key verifies it. This approach offers a substantially stronger security model than HS256, as separate keys are used for signing and verification.

B. bcrypt Password Hashing and the Role of ROD-256

User passwords are hashed with bcrypt before being stored in the database. The adaptive work factor mechanism of bcrypt provides systematic resistance to hardware acceleration. As a result, raw passwords are never stored; sensitive data remains protected even in the event of a database breach. It is important to note that ROD-256 does not replace bcrypt for password storage; ROD-256's primary function in the system is to hash audit ledger blocks. bcrypt remains the industry standard for password security.

C. Hash-Chained Audit Ledger

Every registration and login event is appended to the audit_ledger table as a block. Each block contains a block index, username, event type, status, IP address, timestamp, the digest of the preceding block, and the current block's digest produced by ROD-256. This structure is not a blockchain but rather a hash-chained audit mechanism in which each record carries the digest of its predecessor; it operates over a centralized database and is not fully decentralized. When any block's content is modified, the verification of all subsequent blocks fails, enabling tamper detection.

D. Threat Model

A brief threat model is presented to clarify the security scope of the system. What the system protects: user passwords are protected against brute-force attacks via bcrypt; session tokens are signed with RS256 to guard against impersonation attacks; audit ledger entries allow tamper detection through hash chaining. What the system does not guarantee: collision resistance and preimage resistance of ROD-256 have not been cryptographically proven; the record history can be altered with direct access to the centralized database; ROD-256 cannot replace bcrypt or SHA-256 in production environments. Explicitly defining these boundaries aligns with the educational focus of the project and presents security expectations within a realistic framework.

E. Database Security

Supabase PostgreSQL provides TLS-encrypted connections, row-level security policies, and automated backup mechanisms. The system can automatically fall back to local JSON storage when the DATABASE_URL environment variable is not defined.

VI. FRONT-END IMPLEMENTATION AND USER EXPERIENCE

The front-end developed with React/Vite consists of four main pages: Home, Console, Algorithm, and Dashboard. A glassmorphism UI language and coffee theme form the visual basis, with light/dark theme support implemented via Bootstrap 5.

A. Live Algorithm Visualization

The Algorithm page renders the five mixing steps of ROD-256 in real time. As the user types a message and salt, animated coffee cups visually reflect the changes in the eight internal state values. The interface presents each step—byte conversion, state mixing, rotation, and digest generation—in separate visual panels.

B. Hash Comparison Dashboard

The Live Cryptographic Comparison feature simultaneously displays SHA-256, RIPEMD-160, and SHA3-256 outputs for the same message and salt pair. Users can directly observe algorithm-specific characteristics and output differences.

C. User Dashboard

The Dashboard for authenticated users displays JWT token details, block integrity verification results, and the audit ledger hash-chain status in real time.

VII. STRUCTURAL COMPARISON OF HASH ALGORITHMS

The system performs a structural comparison between ROD-256 and algorithms from the SHA and RIPEMD families. Table I summarizes the principal structural characteristics of the supported algorithms; quantitative performance and avalanche effect measurements are planned for future work.

Algorithm	Output	Structure / Feature
ROD-256	256 bit	ARX + Cyclic state, salt mandatory
SHA-256	256 bit	Merkle-Damgard, 64 rounds
SHA3-256	256 bit	Sponge (Keccak), 1600-bit state
SHA-512	512 bit	Merkle-Damgard, 80 rounds
RIPEMD-160	160 bit	Dual parallel Merkle-Damgard
SHA-1	160 bit	Merkle-Damgard (broken)

Table I. Structural Comparison of Supported Hash Algorithms

ROD-256 distinguishes itself from general-purpose SHA algorithms through its mandatory salt parameter and multi-stage cross-state diffusion. Nevertheless, since it has not yet undergone formal cryptographic verification, it is recommended for educational use only.

VIII. API ENDPOINTS AND BACK-END STRUCTURE

The RESTful API, built on Express.js, consists of three router groups: authentication (authRoutes), audit ledger (ledgerRoutes), and cryptography (cryptoRoutes). The principal endpoints are: POST /api/auth/register, POST /api/auth/login, POST /api/auth/verify-token, POST /api/rod256/hash, POST /api/crypto/compare, GET /api/ledger, and GET /api/ledger/verify.

The system dynamically detects the presence of the DATABASE_URL variable and automatically switches between PostgreSQL cloud storage and local JSON storage. This design enables seamless operation in both full cloud deployment and local development environments.

IX. CLOUD DEPLOYMENT

The project follows a multi-tier deployment architecture distributed across three distinct cloud platforms. Vercel serves the front-end, Render hosts the back-end, and Supabase provides the PostgreSQL database service. The back-end build command (cd backend && npm install && npm run build) compiles the C++ ROD-256 source code with the g++ compiler in a platform-independent manner, producing a .exe binary on Windows and an ELF binary on Render's Linux environment.

X. DISCUSSION

A. Strengths

The ROD-256 system stands out in several technical respects. Each step of the five-stage mixing round applies a distinct diffusion mechanism (XOR, modular addition, variable rotation), aiming to make the output dependent on both the input and the round index. The integration of systems programming with modern cloud development presents an original approach, while the hash-chained audit ledger adds traceability to security records.

B. Limitations

The most significant limitation of ROD-256 is that it has not yet been subjected to formal cryptographic analysis. Evaluations of collision resistance, preimage resistance, and resistance to differential cryptanalysis have not been conducted. Furthermore, the serial processing model was not designed to compete with the throughput of SHA-256 hardware acceleration operating over parallel data paths.

XI. RESEARCH RECOMMENDATIONS

Based on the findings of this study, the following recommendations are highlighted:

1. ROD-256 should be subjected to formal cryptographic analysis covering collision resistance, preimage resistance, and differential attack resistance.
2. The avalanche effect should be measured statistically, computing how many output bits are affected by a single input bit flip.
3. The system should be extended with additional security layers such as multi-factor authentication (MFA), email verification, and account lockout.
4. A refresh token mechanism should be added and JWT lifecycle management strengthened to protect against token theft.

5. The hash-chained audit ledger should be integrated into a decentralized architecture to reduce single-point-of-failure risk.
6. A standalone JavaScript/WebAssembly library should be developed and published on NPM to facilitate reuse of ROD-256 in other systems.
7. Comprehensive performance and stress tests should be conducted to validate the system's resilience and scalability under distributed load conditions.

XII. CONCLUSION

This paper presents a prototype system that integrates an experimental hash algorithm developed in C++ (ROD-256) into a modern cloud-based authentication architecture. The five-stage mixing round of ROD-256—initial XOR combination, left rotation, neighboring state addition, distant state XOR diffusion, and double-layer update—are applied together, targeting a multi-dimensional diffusion structure.

The publicly deployed system (<https://rod-256-auth-system.vercel.app>) creates an interactive learning environment for researchers and students through a live hash visualization, structural comparison dashboard, and verifiable audit ledger. This work provides an inspectable and extensible educational model for developers who wish to design custom cryptographic components for teaching environments.

ACKNOWLEDGMENTS

This work was developed within the Systems Programming course at the Department of Computer Engineering, Kastamonu University. The author sincerely thanks the faculty members of the department for their academic support and guidance. Gratitude is also extended to the Node.js, React, and Supabase open-source communities for the tools used in this project.

REFERENCES

- [1] P. C. van Oorschot and M. J. Wiener, "Parallel collision search with cryptanalytic applications," *Journal of Cryptology*, vol. 12, no. 1, pp. 1-28, 1999. <https://doi.org/10.1007/PL00003816>
- [2] National Institute of Standards and Technology (NIST), "Secure Hash Standard (SHS)," Federal Information Processing Standards Publication 180-4, 2015. <https://doi.org/10.6028/NIST.FIPS.180-4>
- [3] N. Provos and D. Mazieres, "A future-adaptable password scheme," *USENIX Annual Technical Conference*, pp. 81-91, 1999.
- [4] C. Percival and S. Josefsson, "The scrypt password-based key derivation function," RFC 7914, IETF, 2016. <https://doi.org/10.17487/RFC7914>
- [5] A. Biryukov, D. Dinu, and D. Khovratovich, "Argon2: Memory-hard function for password hashing," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 3747-3758, 2021. <https://doi.org/10.1109/TIFS.2021.3111026>
- [6] S. Bhatt and R. K. Patil, "Multi-factor authentication in securing web applications against cyber threats," *Int. Journal of Computer Science and Information Security*, vol. 17, no. 4, pp. 45-52, 2019.
- [7] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015. <https://doi.org/10.17487/RFC7519>
- [8] M. Jones and J. Hildebrand, "JSON Web Encryption (JWE)," RFC 7516, IETF, 2015. <https://doi.org/10.17487/RFC7516>
- [9] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. <https://bitcoin.org/bitcoin.pdf>
- [10] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8th ed. Hoboken, NJ: Pearson, 2022.

- [11] B. Schneier, *Applied Cryptography: Protocols, Algorithms and Source Code in C*, 2nd ed. New York: Wiley, 1996.
- [12] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, IETF, 2010. <https://doi.org/10.17487/RFC5869>
- [13] Supabase Inc., "Supabase documentation," 2024. [Online]. <https://supabase.com/docs>
- [14] Vercel Inc., "Vercel platform documentation," 2024. [Online]. <https://vercel.com/docs>
- [15] R. A. Mukhtar, "ROD-256 Auth System - GitHub Repository," 2025. [Online]. <https://github.com/RodinaAmrMukhtar/rod-256-auth-system>